playkit-js-aws-analytics

Complete Plugin Guide

Kaltura Player JS Plugin · v1.2.0

Bridging Kaltura Player analytics to the AWS TV Libra framework

12 analytics events · 3 CustomEvent channels · Full E2E test coverage

September 2026

Table of Contents

18. Important Considerations

Part I

Overview — For Everyone

1. What This Plugin Does

The **playkit-js-aws-analytics** plugin is a bridge between the **Kaltura Video Player** and the **AWS TV Libra analytics framework**.

When a viewer watches a video on a page powered by the Kaltura Player, this plugin automatically detects what the viewer does — pressing play, pausing, seeking, changing volume, going fullscreen, watching past a threshold, completing the video, and more — and translates each of those actions into the exact analytics events that the AWS TV analytics infrastructure expects.

The plugin is a **drop-in replacement** for the analytics layer in the existing AWS TV video player. The AWS TV team can switch from their current player to the Kaltura Player without losing any analytics data, dashboards, or downstream integrations.

> **Key Benefit**
> Zero analytics disruption when migrating to Kaltura Player. Every event, every payload field, every CustomEvent channel is replicated exactly.

2. Why It Exists

The AWS TV video player currently uses a custom-built analytics layer that sends events to the **Libra framework** via browser `CustomEvent` dispatch. Downstream systems — dashboards, engagement

metrics, content recommendations, and operational monitoring — all depend on receiving these specific events in a specific format.

When migrating to the Kaltura Player, these analytics must continue working identically. Rather than requiring the AWS TV team to rewire their entire analytics pipeline, this plugin ensures the Kaltura Player speaks the same language.

3. What It Tracks

The plugin tracks **12 distinct analytics events** across three categories:

Playback Events (Automatic)

EVENT	WHEN IT FIRES	FIRES ONCE?
Play	The very first time the viewer presses play	Yes
Pause	The viewer pauses the video (not during seeking or at video end)	Yes
Replay	The viewer plays the video again after it has ended	Yes
Watch	The viewer has watched at least 30 seconds of actual content	Yes
Complete	The viewer has watched 95% or more of the video	Yes
WatchTime	The viewer leaves the page or switches tabs (captures total seconds watched)	Per cycle*
VolumeChange	The viewer changes the volume	Yes
FullScreenChange	The viewer enters or exits fullscreen	Yes

** WatchTime fires once each time the page becomes hidden, resets when the page becomes visible again, and fires again next time it's hidden.*

UI-Triggered Events (Manual)

EVENT	WHEN IT FIRES	FIRES ONCE?
SeekForward	Custom UI button triggers a forward seek	Yes
SeekBackward	Custom UI button triggers a backward seek	Yes
TranscriptToggle	User toggles the transcript panel	Yes
QualityChange	User changes video quality	Yes

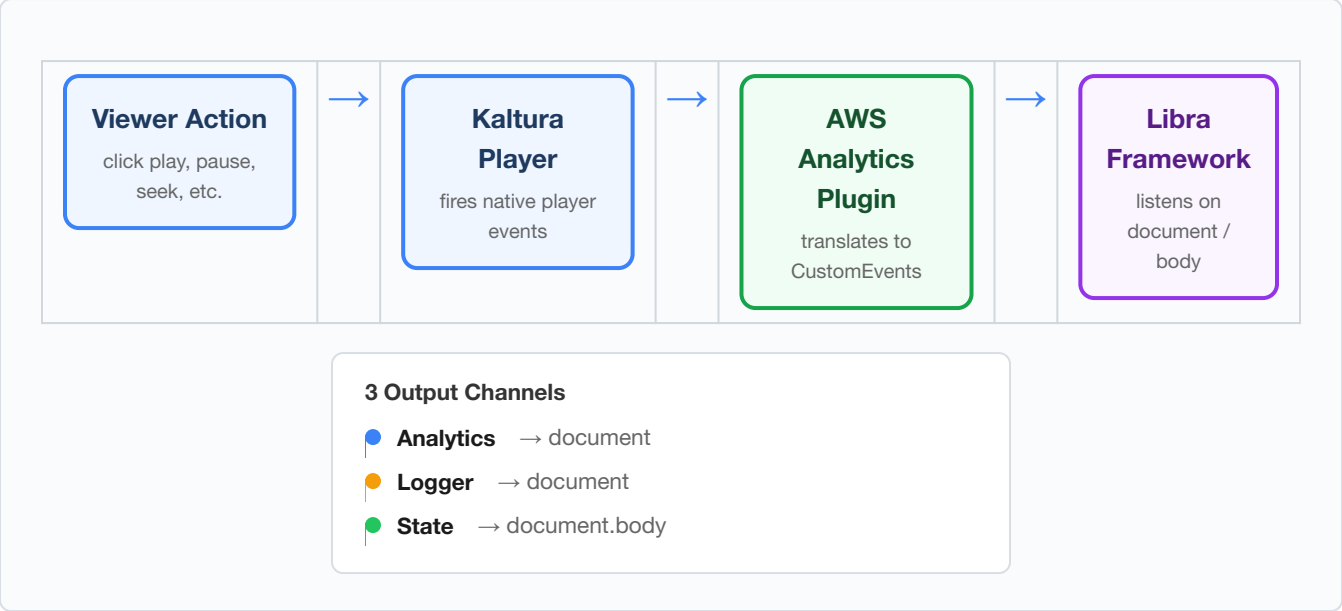
Operational Metrics (Automatic)

METRIC	WHEN IT FIRES	FIRES ONCE?
VideoBufferTime	Every time the player exits a buffering state (captures duration in seconds)	No (fires each time)
VideoPlayerReady	When the player reaches `CAN_PLAY` state (captures time-to-ready in milliseconds)	Yes (fires once)
VideoError	On any player error (captures code, message, videoId, videoTitle)	No (fires each time)

Player State Broadcasts (Automatic)

STATE	WHEN IT FIRES	FIRES ONCE?
play	Every time playback starts or resumes	No (fires each time)
pause	Every time playback pauses, and just before `ended`	No (fires each time)
ended	Video reaches the end	No (fires each time)

4. How Data Flows



The plugin sits between the Kaltura Player's internal event system and the browser's `CustomEvent` API. It listens for native player events (play, pause, timeupdate, etc.), applies business logic (deduplication, thresholds, seek filtering), and dispatches `CustomEvent` objects that the Libra framework's existing listeners pick up unchanged.

5. Configuration Options

OPTION	TYPE	DEFAULT	DESCRIPTION
`videoId`	string	*auto*	**Auto-resolved** from `player.sources.id` (the Kaltura entry ID). Set explicitly only if you need a custom override.
`videoTitle`	string	*auto*	**Auto-resolved** from `player.sources.metadata.name` (the entry's title in KMC). Set explicitly only if you need a custom override.
`orgId`	string	`'awsm_tv'`	Organization identifier sent in buffer-time logger events. Change this if your Libra configuration uses a different org ID.
`watchThresholdSeconds`	number	`30`	How many seconds of actual watch time before the `Watch` event fires. Based on the HTML5 `played` TimeRanges, so seeking doesn't inflate the count.
`completeThresholdPercent`	number	`0.95`	Fraction (0–1) of total video duration that must be watched before `Complete` fires. 0.95 means 95%.

Zero-Config for Most Use Cases

`videoId` and `videoTitle` are automatically pulled from the Kaltura Player's media info after `loadMedia()`. In the typical case, the only config you need is an empty `awsAnalytics: {}` object to activate the plugin. Explicit overrides are available when you need a custom video ID or title that differs from the Kaltura entry metadata.

Why No Endpoint URLs or API Keys?

This plugin is a **CustomEvent bridge**, not an HTTP transport. It dispatches browser events that the Libra framework (already loaded on the AWS TV page) listens for. Libra handles all network transport, authentication, batching, and retries. The plugin has zero network dependencies of its own.

Part II

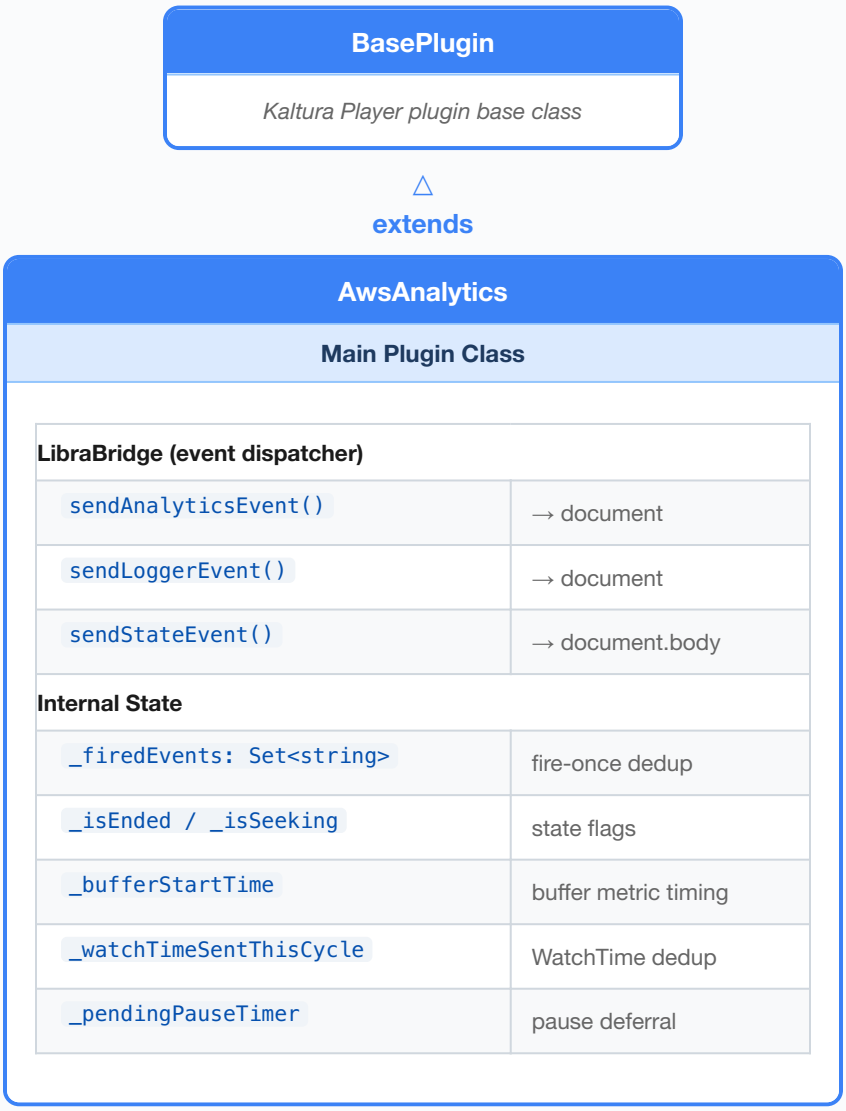
Technical Reference

6. Architecture & Source Files

The plugin consists of 4 source files totaling ~280 lines of TypeScript:

FILE	PURPOSE	LINES
`src/index.ts`	Entry point. Registers the plugin with the Kaltura Player via `registerPlugin()`.	4
`src/aws-analytics.ts`	Main plugin class. Extends `BasePlugin`. Contains all event handlers, thresholds, dedup logic, and page lifecycle management.	245
`src/libra-bridge.ts`	Thin wrapper that dispatches `CustomEvent` objects on the correct DOM targets and channel names.	19
`src/types/index.ts`	TypeScript interfaces for config, event payloads, and the `UiEventName` union type.	34

Class Diagram



7. Event Implementation Details

7.1 Play

Listens to: `FIRST_PLAYING`

The Kaltura Player fires `FIRST_PLAYING` exactly once per media load, so this maps directly. Also sends a `play` state broadcast.

7.2 Pause

Listens to: `PAUSE`
This is the most complex event due to false positives. The Kaltura Player fires `PAUSE` during seeks and at video end. The plugin uses a **deferred dispatch** pattern:

1. On `PAUSE`, schedule a `setTimeout(0)` callback
2. When the callback runs, check if `_isSeeking` or `_isEnded` is true
3. If either is true, suppress the Pause analytics event
4. The state broadcast (`state: pause`) fires immediately (not deferred) for user-initiated pauses, but is suppressed during seek and at video end (where `_onEnded` handles it instead)

7.3 Replay

Listens to: `PLAYING`
When `PLAYING` fires and `_isEnded` is `true`, this means the viewer is watching again after the video ended. The plugin fires the `Replay` event and resets `_isEnded`.

7.4 Watch & Complete

Listens to: `TIME_UPDATE`
On every time update, the plugin calculates actual watched duration from the HTML5 `video.played` TimeRanges (which excludes seeked-over segments). If the duration exceeds `watchThresholdSeconds`, the `Watch` event fires. If the ratio of watched-to-total exceeds `completeThresholdPercent`, the `Complete` event fires with the total duration.

7.5 WatchTime

Listens to: `visibilitychange`, `pagehide`, `beforeunload`
Captures total seconds watched and sends them when the user leaves the page or tabs away. Uses `_watchTimeSentThisCycle` to prevent duplicate sends within a single hidden-cycle, and resets when the tab becomes visible again.

7.6 VolumeChange & FullScreenChange

Listens to: `VOLUME_CHANGE`, `ENTER_FULLSCREEN`, `EXIT_FULLSCREEN`
Straightforward fire-once events with no additional logic.

7.7 SeekForward & SeekBackward

Listens to: `SEEKING`, `SEEKED`
On `SEEKING`, the current position is recorded. On `SEEKED`, the delta is calculated. If the delta exceeds 1 second forward or backward, the corresponding event fires. The `sendUiEvent()` API is also available as a secondary trigger for custom UI controls.

7.8 QualityChange & TranscriptToggle

Listens to: `VIDEO_TRACK_CHANGED`, `TEXT_TRACK_CHANGED`
Auto-detected from native player events when the video quality or caption/subtitle track changes. Also available via `sendUiEvent()` for custom UIs.

7.9 VideoBufferTime (Logger Metric)

Listens to: `PLAYER_STATE_CHANGED`
When the player enters `BUFFERING` state, a timestamp is recorded. When it exits, the duration is calculated and sent on the logger channel. This fires every time buffering occurs (not fire-once).

7.10 VideoPlayerReady (Logger Metric)

Listens to: `CAN_PLAY`
Measures the time from plugin initialization to when the player can begin playback. Sent once as milliseconds on the logger channel.

7.11 VideoError (Logger Metric)

Listens to: `ERROR`
Captures player errors with code, message, videoId, and videoTitle. Sent on the logger channel with `logLevel: "error"`. Fires on every error (not fire-once).

7.12 State Broadcasts

State events (`play`, `pause`, `ended`) fire on **every occurrence**, not fire-once. At natural video end, the plugin explicitly sends `pause` then `ended` in sequence to match the real AWS TV player's ordering.

8. CustomEvent Channel Reference

CHANNEL NAME	DOM TARGET	PURPOSE	EVENTS SENT
`custom_aws_eb-analytics_notify-listener`	`document`	Primary analytics events consumed by Libra	All 12 analytics events (Play, Pause, Watch, WatchTime, etc.)
`custom_aws_eb-logger_notify-listener`	`document`	Operational metrics (performance, errors)	VideoBufferTime, VideoPlayerReady, VideoError
`custom_awstv_video-player-state`	`document.body`	Real-time player state for UI synchronization	play, pause, ended

Important: DOM Target Difference

The analytics and logger channels dispatch on `document`, but the state channel dispatches on `document.body`. This matches the real AWS TV player's behavior. Listeners must be attached to the correct target.

9. Payload Schemas

Analytics Event Payload

```
{
  "type": "awsTvVideo",           // always this literal
  "name": "Play",                 // event name
  "videoId": "1_abc123",          // auto-resolved or from config override
  "videoTitle": "My Video",       // auto-resolved or from config override
  // Optional fields (event-specific):
  "duration": "574",              // Complete event only — rounded seconds
  "timestamp": "1714000000000",   // WatchTime only — Date.now() as string
  "watchTime": "145",             // WatchTime only — rounded seconds watched
}
```

Logger Event Payload (Buffer Metric)

```
{
  "logLevel": "info",
  "namespace": "VideoPlayer",
  "orgId": "awsm_tv",            // from config.orgId (default: "awsm_tv")
  "metricName": "VideoBufferTime",
  "payload": {
    "duration": 1.234,           // seconds as number
    "unit": "Seconds"
  }
}
```

State Broadcast Payload

```
{
  "state": "play",               // "play" | "pause" | "ended"
  "videoId": "1_abc123"          // from config
}
```

10. Edge Case Handling

SCENARIO	BEHAVIOR	HOW IT WORKS
Seek triggers PAUSE event	Pause analytics event suppressed	`_isSeeking` flag is set on `SEEKING`, cleared on `SEEKED`. The deferred Pause handler checks this flag.
Video ends naturally (triggers PAUSE then ENDED)	Pause analytics event suppressed; state broadcasts fire as `pause` then `ended`	`_onEnded()` sets `_isEnded=true` and sends both state events. `_onPause()` checks `_isEnded` to suppress duplicates.
Viewer seeks without playing (scrubbing)	No Watch/Complete inflation	Watch time is calculated from `video.played` TimeRanges, which only counts actual playback, not seeked-over segments.
Rapid tab switching	WatchTime fires once per hidden cycle	`_watchTimeSentThisCycle` prevents double-send. Resets when tab becomes visible.
New media loaded	All fire-once events can fire again	`loadMedia()` clears `_firedEvents`, resets all flags, clears pending timers.
Player duration is null (live or loading)	Complete event safely skipped	`duration ?? 0` null-coalescing ensures no division-by-zero or NaN.

Part III

Integration & Deployment

12. Installation & Setup

Option A: Script Tag (Recommended for AWS TV)

```
<!-- 1. Load the Kaltura Player -->
<script src="https://cdn.jsdelivr.net/npm/@playkit-js/kaltura-player-js@latest/dist/kaltura-ovp-playe

<!-- 2. Load the plugin (after the player) -->
<script src="playkit-aws-analytics.js"></script>

<!-- 3. Set up the player — zero-config (videoId and videoTitle auto-resolve) -->
<script>
  const player = KalturaPlayer.setup({
    targetId: 'player-container',
    provider: {
      partnerId: YOUR_PARTNER_ID,
      uiConfId: YOUR_UICONF_ID
    },
    plugins: {
      awsAnalytics: {}   // videoId & videoTitle auto-resolve from the entry
    }
  });
  player.loadMedia({ entryId: '1_abc123' });
</script>
```

With Explicit Overrides

```
plugins: {
  awsAnalytics: {
    videoId: 'custom-id',           // overrides player.sources.id
    videoTitle: 'Custom Title',     // overrides player.sources.metadata.name
    orgId: 'my_org',               // overrides default 'awsm_tv'
    watchThresholdSeconds: 15,      // fire Watch after 15s instead of 30s
    completeThresholdPercent: 0.90  // fire Complete at 90% instead of 95%
  }
}
```

Option B: jsDelivr from GitHub

This plugin is not published to npm. It's distributed straight from the GitHub repo via jsDelivr, which serves any tagged commit's files directly:

```
<script src="https://cdn.jsdelivr.net/gh/kaltura/playkit-js-aws-analytics@v1.2.0/dist/playkit-aws-ana
```

Pin to a specific tag (as above) in production — tags are `v`-prefixed. `@latest` works too, but only for quick testing since it can silently move to a newer release.

> **Auto-Registration**
>
> The plugin registers itself automatically when the script loads. No manual registration code is needed. Just include it in your player config's `plugins` object.

13. UI Event API

Four events cannot be detected from native player events and must be triggered by the host application's custom UI controls:

```
// Get the plugin instance
const plugin = player._pluginManager.get('awsAnalytics');

// Call sendUiEvent with one of the four allowed event names:
plugin.sendUiEvent('SeekForward');
plugin.sendUiEvent('SeekBackward');
plugin.sendUiEvent('TranscriptToggle');
plugin.sendUiEvent('QualityChange');
```

These are type-safe — TypeScript will only accept the four valid event names:

```
type UiEventName = 'SeekBackward' | 'SeekForward' | 'TranscriptToggle' | 'QualityChange';
```

Like all other analytics events, UI events are **fire-once per media session**. Calling `sendUiEvent('SeekForward')` multiple times will only dispatch the CustomEvent the first time.

14. Listening to Events (Consumer Side)

The Libra framework (or any consumer) listens for these events using standard DOM event listeners:

```
// Analytics events (all 12)
document.addEventListener('custom_aws_eb-analytics_notify-listener', (e) => {
  console.log('Analytics:', e.detail);
  // { type: 'awsTvVideo', name: 'Play', videoId: '...', videoTitle: '...' }
});

// Buffer metrics
document.addEventListener('custom_aws_eb-logger_notify-listener', (e) => {
  console.log('Logger:', e.detail);
  // { logLevel: 'info', namespace: 'VideoPlayer', ... }
});

// Player state (note: document.body, not document)
document.body.addEventListener('custom_awstv_video-player-state', (e) => {
  console.log('State:', e.detail);
  // { state: 'play', videoId: '...' }
});
```

15. Development Workflow

COMMAND	WHAT IT DOES
`npm run serve`	Starts webpack dev server with the demo page at `http://localhost:8080`
`npm run build`	Production build → `dist/playkit-aws-analytics.js` (minified UMD, ~11.1 KB)
`npm run type-check`	Runs TypeScript compiler in check-only mode
`npm run lint`	ESLint + Prettier check
`npm test`	Full pipeline: build → copy bundle → Cypress E2E (37 tests in Chrome)
`npm run test:watch`	Same as test but opens Cypress interactive runner

16. Test Coverage

The Cypress E2E test suite contains **37 tests** running against a real Kaltura Player instance with a real video entry. Every test loads the player, performs real playback actions, and asserts on captured CustomEvents.

CATEGORY	# TESTS	WHAT'S COVERED
Play	2	First play fires once, doesn't re-fire on pause/resume
Pause	4	User pause, seek suppression, end suppression, fire-once dedup
VolumeChange	1	Fire-once across multiple changes
FullScreenChange	1	Fire-once dedup
UI Events	2	All 4 UI events fire once each
State Broadcasts	4	Play repeats, ended fires, pause-before-ended ordering, seek suppression
Replay	2	Fires after ended, doesn't fire on normal resume
Watch Threshold	3	Fires at threshold, doesn't fire before, correct payload
Complete Threshold	3	Fires at threshold, includes duration, doesn't fire below
WatchTime	3	Fires on hidden, no double-fire, re-fires after visible cycle
loadMedia Reset	3	Fire-once resets, pending pause cleared, WatchTime tracking resets
Payload Schema	2	Base fields correct, state events include videoId
Buffer Metrics	1	Logger channel schema validation (when buffering occurs)
Plugin Instantiation	2	Plugin accessible, config merged with defaults (incl. orgId)
Auto-Resolve	4	videoId from sources, videoTitle from metadata, config override priority, state broadcast auto-resolve

> **37 / 37 Tests Passing**
>
> All tests pass against a live Kaltura OVP endpoint with a real video entry. The test suite runs in ~4.5 minutes on Chrome.

17. Production Deployment Checklist

1. **Build the production bundle:**

```
npm run build
```

This produces `dist/playkit-aws-analytics.js` (~11.1 KB minified) and its source map.

2. **Run the full test suite:**

```
npm test
```

Ensure 37/37 tests pass.

3. **Deploy the bundle file** to your CDN or static asset server alongside the Kaltura Player script.
4. **Add the script tag** to your page, after the Kaltura Player script.
5. **Add the plugin config** to your `KalturaPlayer.setup()` call. An empty `awsAnalytics: {}` is sufficient — videoId and videoTitle auto-resolve from the Kaltura entry metadata. Override only if needed.
6. **Wire up UI events** in your custom controls (seek buttons, transcript toggle, quality selector) by calling `plugin.sendUiEvent()`.
7. **Verify in production:** Open browser DevTools, go to the Console, and paste:

```
document.addEventListener('custom_aws_eb-analytics_notify-listener',
  (e) => console.log('Analytics:', e.detail));
```

Play the video and confirm events appear.

18. Important Considerations

18.1 Fire-Once Semantics

All analytics events (except WatchTime) fire **at most once per media session**. When `loadMedia()` is called (e.g., switching to a new video), all fire-once tracking resets. This matches the original AWS TV player's behavior exactly.

18.2 State Broadcast Ordering at Video End

When a video ends naturally, the plugin sends `state: pause` immediately followed by `state: ended`. This matches the exact ordering observed in the production AWS TV player (confirmed via browser captures). Downstream consumers that depend on seeing a `pause` before `ended` will continue working.

18.3 WatchTime and Page Lifecycle

WatchTime uses three mechanisms to capture data before the page unloads: `visibilitychange`, `pagehide`, and `beforeunload`. This triple-listener approach maximizes reliability across browsers. The `CustomEvent` dispatch is synchronous, so the data is captured even during rapid tab/page closure.

18.4 Watch Duration Accuracy

Watched time is calculated from the HTML5 `video.played` TimeRanges, which tracks only actual playback segments. Seeking over content does not inflate the watch count. This is more accurate than simple elapsed-time tracking.

18.5 Plugin Access Pattern

The plugin is accessed at runtime via `player._pluginManager.get('awsAnalytics')`. While `_pluginManager` is technically an internal API, this is the standard pattern used by all Kaltura playkit-js plugins and is stable across player versions.

18.6 Bundle Size

The production bundle is **~11.1 KB** minified (with source map). It has **zero runtime dependencies** beyond the Kaltura Player itself. The only external reference is `BasePlugin` from `@playkit-js/kaltura-player-js`, which is resolved as a webpack external (the player is already loaded on the page).

18.7 Browser Compatibility

The plugin is compiled via Babel with the Kaltura Player's shared browserslist config, targeting the same browsers as the player itself. It uses only standard DOM APIs (`CustomEvent`, `EventTarget`, `TimeRanges`, `visibilitychange`) available in all modern browsers.

18.8 Auto-Resolution of videoId and videoTitle

The plugin reads `player.sources.id` and `player.sources.metadata.name` at the moment each event fires (not at construction time). This means:

- After `loadMedia()`, the values update automatically for the new entry
- No manual config update is needed when switching videos in a playlist
- Explicit config values always take priority over auto-resolved values

18.9 CI/CD

A single GitHub Actions workflow (`ci.yaml`) runs on every pull request and push to `main` :

- **build-and-lint:** Build, type-check, ESLint/Prettier, and `npm audit`
- **e2e:** Full 37-test Cypress suite in real Chrome, against a live Kaltura entry (needs `build-and-lint`)
- **deploy-pages:** Publishes `docs/` to GitHub Pages (push to `main` only), gated on both prior jobs passing

There is no npm publish step — the plugin is never published to the npm registry. Releases are cut with `npm run release` (bumps version, builds `dist/` , commits, tags), and jsDelivr serves the tagged commit's `dist/` directly from GitHub.
